

Estado de caja — fórmulas de cálculo y ubicación en el código

fontana_movimientos · movimientos_caja/views.py

Rediseño del 28/09/2026 — documento de referencia técnica

1. Qué cambió y por qué

Hasta el 25/09/2026, "Estado de caja" calculaba el saldo de Macro y de Nación a partir del saldo_inicial del ÚLTIMO libro (el "vigente") de cada caja, sumándole sólo los movimientos cargados en ESE mismo libro. Al crear el libro 3 del Macro, esto dejó de funcionar: un movimiento cargado en un libro viejo, si tenía un diferido que recién se cumplía más adelante, o era un cheque en cartera que se cobraba después, desaparecía del cálculo el día que se resolvía.

Después de dos correcciones intermedias, la aclaración decisiva fue tuya: el saldo_inicial de un libro NUEVO ya es el arrastre de TODO lo anterior (firme o no) — "es sólo el arrastre para saber qué valor histórico viene teniendo", nunca una base limpia para seguir sumando. Por eso el diseño final abandona por completo la idea de "libro vigente" como base de cálculo:

saldo_a_fecha = saldo_inicial del PRIMER libro de la caja
+ TODOS los movimientos firmes de la caja, de
CUALQUIER libro

El "libro vigente" (el más nuevo) sigue existiendo y se sigue mostrando en pantalla — es donde se cargan los movimientos nuevos — pero ya NO participa en la cuenta. Un libro nunca se "cierra": es sólo una hoja más de una misma seguidilla de movimientos de esa caja.

2. Convención de signos

Todos los montos (saldo_inicial, monto de cada movimiento, saldo mostrado) usan siempre la misma convención, cargada así desde que se ingresan los datos — el sistema nunca invierte signos en ningún paso del cálculo:

- Positivo = le debemos al banco (deuda).
- Negativo = a favor nuestro.

Por eso un saldo negativo grande, como el de tu pantalla, es una buena noticia: significa que el banco (o los libros de Macro/Nación en conjunto) están a favor de la empresa.

3. Los tres bloques del cálculo

3.1 — Saldo de origen (la base)

Función: `_primer_libro_de_caja(caja)` — archivo `movimientos_caja/views.py`

Busca, de todos los LibroCaja que tiene esa caja, el de fecha_creacion MÁS ANTIGUA (si hay empate o falta la fecha, desempata por id). Su saldo_inicial es el único saldo_inicial de todos los libros que sirve como base de cálculo — es el momento en que esa caja no tenía ningún movimiento cargado todavía, así que no puede tener nada "adentro" ya contado.

```
def _primer_libro_de_caja(caja):
    return (
        LibroCaja.objects.filter(caja=caja)
        .order_by(F('fecha_creacion').asc(nulls_first=True), 'id')
        .first()
    )
```

Por qué el saldo_inicial de los demás libros NO sirve de base

El saldo_inicial de un libro nuevo (libro 2, libro 3, ...) es el arrastre del saldo a la fecha en que se lo creó — y ese arrastre YA tiene adentro absolutamente todos los movimientos anteriores, se hayan cobrado/vencido o no. Volver a sumarle movimientos de libros viejos a ESE saldo (como hacía la primera versión de este fix) duplicaba montos que ya estaban adentro. Por eso el sistema usa siempre el saldo del PRIMER libro (el único sin nada adentro todavía) como única base, y a partir de ahí construye todo el saldo actual sumando movimientos reales, uno por uno.

3.2 — Movimientos firmes de la caja

Función: **`_movimientos_firmes_de_caja(caja, fecha)`**

Un movimiento se considera FIRME a una fecha si:

- no tiene diferido cargado (se cobra/paga de inmediato), o
- tiene diferido, pero esa fecha de diferido ya llegó (diferido $\leq$ fecha elegida).

Se suman los de CUALQUIER libro de esa caja (no sólo el vigente), porque un libro no es un período cerrado. Se excluyen únicamente los movimientos que todavía no tienen NINGÚN libro asignado (esos se avisan aparte en pantalla como advertencia — igual que ya pasaba antes de este rediseño).

```
def _movimientos_firmes_de_caja(caja, fecha):
    return MovimientoCaja.objects.filter(
        caja=caja, asiento_libro__isnull=False,
    ).filter(
        Q(movimientocajadiferido__isnull=True)
        | Q(movimientocajadiferido__diferido__isnull=True)
        | Q(movimientocajadiferido__diferido__lte=fecha)
    )
```

3.3 — Cheques en cartera

Funciones: **`_cheques_en_cartera_qs(caja)`** y **`_cheques_en_cartera(caja)`**

Son los movimientos de tipo "Cheque", concepto "Cartera", sin diferido y TODAVÍA SIN EFECTIVIZAR (efectivización vacía) de esa caja puntual. NO son cheques que la empresa recibió: son cheques que Fontana tiene preparados para ENTREGARLE a un productor, y que todavía no se le dieron. Por eso el monto se carga en positivo al principio (como cualquier pago que sale) — pero mientras el cheque sigue "en cartera" (sin entregar), esa plata en los hechos sigue siendo de la empresa.

```
def _cheques_en_cartera_qs(caja):
    return (
        MovimientoCaja.objects.filter(
            caja=caja, tipo__nombre__iexact='Cheque',
```

```

        rel_concepto__concepto_tipo__nombre__iexact='Cartera',
        efectivizacion__isnull=True,
    )
    .filter(Q(movimientocajadiferido__isnull=True)
            | Q(movimientocajadiferido__diferido__isnull=True))
)

```

Corrección 28/09/2026 — el signo de la cartera pendiente

Mientras un cheque sigue en cartera (sin diferido, sin efectivizar) cuenta A FAVOR nuestro: se RESTA del saldo, no se suma — aunque su monto esté guardado en positivo. En palabras de Gastón: "esos cheques al no tener aún diferido, estar en cartera y no tener efectivización es como que aún no se entregaron a productores, así que debería ser a favor nuestro hasta que se entreguen".

Apenas el cheque tiene una fecha de diferido (se programa la entrega) o se efectiviza (se entrega/cobra), deja de estar "en cartera" y pasa a contarse en el renglón de "Movimientos" con su monto guardado SIN invertir — recién ahí es una deuda real. El dato guardado en la base nunca cambia; sólo cambia, según el estado del cheque, si se lo suma o se lo resta al armar el saldo.

Función: **`_cheques_en_cartera(caja)`** devuelve la suma YA con este ajuste aplicado (el total negado), lista para sumarse tal cual al saldo:

```

def _cheques_en_cartera(caja):
    total = _cheques_en_cartera_qs(caja)\
        .aggregate(total=Sum('monto'))['total'] or Decimal('0')
    return -total # a favor nuestro mientras sigue sin entregar

```

4. Pantalla "Calcular" genérica (cualquier caja)

Función: `_calcular_estado_caja(caja, fecha)`. Usa los mismos dos bloques que "Calcular por defecto" (sección 5) — movimientos firmes sin la cartera pendiente, más la cartera pendiente ya con su signo corregido — así que las dos pantallas siempre dan el mismo total para la misma caja.

```

saldo_a_fecha = saldo_inicial(primer_libro)
                + _movimientos_firmes_excluyendo_cartera(caja,
fecha)
                + _cheques_en_cartera(caja)           <-- ya viene
negado

```

```

primer_libro = _primer_libro_de_caja(caja)
saldo_inicial = primer_libro.saldo_inicial or Decimal('0')
total_firme = _movimientos_firmes_excluyendo_cartera(caja, fecha) \
    + _cheques_en_cartera(caja)
saldo_a_fecha = saldo_inicial + total_firme

```

5. Pantalla "Calcular por defecto" (Macro, Nación, Global)

Esta pantalla arma el mismo total que la genérica, pero lo muestra desglosado en renglones separados, para que se vea de dónde sale cada parte. Función: `_saldo_base_defecto(caja, fecha)`.

Renglón 1 "Movimientos":

```

monto = _movimientos_firmes_excluyendo_cartera(caja, fecha)
saldo = saldo_inicial(primer_libro) + monto

```

Renglón 2 "Cheques en cartera":

```

monto = _cheques_en_cartera(caja)          <-- negativo
mientras está pendiente
    saldo = saldo del Renglón 1 + monto      <-- ESTE es el
saldo final

```

`_movimientos_firmes_excluyendo_cartera` es igual a `_movimientos_firmes_de_caja`, pero descartando los ids que ya aparecen en `_cheques_en_cartera_qs` (para no sumarlos dos veces: un cheque en cartera no tiene diferido, así que sin este descarte contaría como "firme" en el Renglón 1 Y de nuevo en el Renglón 2).

```

def _movimientos_firmes_excluyendo_cartera(caja, fecha):
    ids_cartera = set(_cheques_en_cartera_qs(caja).values_list('id', flat=True))
    total = Decimal('0')
    for monto, mov_id in _movimientos_firmes_de_caja(caja, fecha)\
        .values_list('monto', 'id'):
        if mov_id not in ids_cartera:
            total += monto
    return total

def _saldo_base_defecto(caja, fecha):
    primer_libro = _primer_libro_de_caja(caja)
    saldo_inicial = primer_libro.saldo_inicial or Decimal('0')
    movimientos_firmes = _movimientos_firmes_excluyendo_cartera(caja, fecha)
    cartera = _cheques_en_cartera(caja)

    saldo_tras_movimientos = saldo_inicial + movimientos_firmes    # Renglón 1
    saldo_tras_cartera = saldo_tras_movimientos + cartera          # Renglón 2 =
    saldo_final

```

El "Saldo actual" grande que se ve arriba de cada tarjeta (Macro / Nación) en la pantalla es exactamente ese `saldo_final` — es decir, YA incluye la cartera sumada una vez (ver la sección 8, tu pregunta puntual).

Para Macro y Nación juntos, "Global" simplemente suma, fecha por fecha, el último saldo conocido de cada una (manteniendo el valor de la que no tuvo movimiento ese día) — no cambia ninguna fórmula, sólo combina los dos totales ya calculados.

6. Proyección a fechas futuras

Función: **`_movimientos_diferidos_pendientes(caja, fecha)`**

Junta, agrupados por día, todos los movimientos de esa caja (cualquier libro) cuyo diferido es POSTERIOR a la fecha elegida — es decir, lo que todavía falta que llegue. A partir del saldo final calculado arriba, cada fecha futura le va sumando su propio monto del día, generando el "saldo proyectado" de cada renglón siguiente. Los diferidos de fecha igual o anterior a la elegida no aparecen acá porque ya están adentro del saldo firme (son errores de carga si aparecieran, no pendientes reales).

```

filas = (
    MovimientoCaja.objects.filter(caja=caja,
        movimientocajadiferido__diferido__gt=fecha)
    .values('movimientocajadiferido__diferido')
    .annotate(total_dia=Sum('monto'))
    .order_by('movimientocajadiferido__diferido')
)

```

Si no hay ningún diferido pendiente a futuro, la proyección queda vacía y "Saldo actual" y "Saldo proyectado" muestran el MISMO número — es lo que se ve en tu pantalla (19.003.489,58 repetido dos veces): no es un error, es la señal de que no queda nada pendiente por resolver más adelante en esa caja a esa fecha.

7. Ejemplo numérico completo

Con números simples, para seguir el cálculo paso a paso (no son tus datos reales, es sólo para mostrar cómo encajan las piezas):

Concepto	Monto	Saldo acumulado
Saldo inicial del PRIMER libro (origen)	-15.000.000,00	-15.000.000,00
Renglón 1 — Movimientos (todos los libros, sin cartera)	-3.200.000,00	-18.200.000,00
Renglón 2 — Cheques en cartera (sin entregar)	-800.000,00	-19.000.000,00 ← saldo_final

El "saldo_final" (-19.000.000,00 en este ejemplo) es lo que se muestra como "Saldo actual" en pantalla. Ya tiene sumados, una sola vez cada uno: el saldo de origen, todos los movimientos firmes, y la cartera pendiente (con su signo ya corregido a favor nuestro — el -800.000,00 del Renglón 2 es justamente lo que devuelve _cheques_en_cartera, no el monto crudo guardado en la base).

8. Tu pregunta: por qué restar la cartera de nuevo al final está mal

Vos tenías armado algo que, al valor final que muestra la pantalla (el negativo, ej. -19.003.489,58), le volvía a restar el monto de la cartera. Seguime el razonamiento con el ejemplo de arriba:

El punto clave

La cartera YA está sumada una vez adentro de ese saldo final (es exactamente el Renglón 2). Volver a tocarla después de leer ese número es operar sobre algo que el sistema ya resolvió — el resultado deja de representar la realidad, pase lo que pase con el signo que uses.

Hay dos formas de "restar la cartera de nuevo", y las dos dan un número incorrecto, aunque por motivos distintos:

8.1 — Si restás el monto de cartera tal cual está cargado (con su signo)

Siguiendo el ejemplo: saldo_final (-19.000.000,00) menos el monto de cartera (-800.000,00) da -19.000.000,00 - (-800.000,00) = -18.200.000,00. Ese resultado es EXACTAMENTE el saldo del Renglón 1 — es decir, deshiciste la suma de la cartera. No lograste "descontar la cartera de más": lograste borrarla del cálculo, como si esos cheques pendientes no existieran. Si tu intención era justamente esa (ver el

saldo SIN contar la cartera todavía pendiente), no hacía falta restar nada: ese valor ya te lo da el sistema directamente en el Renglón 1, sin ningún cálculo extra.

8.2 — Si restás el monto de cartera en positivo (su valor absoluto)

Acá sí hay una duplicación real: $-19.000.000,00 - 800.000,00 = -19.800.000,00$. La cartera terminó restándose DOS veces en la misma dirección — una vez adentro del cálculo del sistema (Renglón 2), y otra vez a mano encima del resultado ya final. El saldo mostrado queda mal por exactamente el doble del monto de la cartera.

Qué hacer en cambio

Si lo que buscás es un número que EXCLUYA la cartera pendiente (por ejemplo, para saber cuánto hay "asegurado" sin contar cheques que todavía no se cobraron), tomá directamente el saldo del Renglón 1 ("Movimientos") — es el mismo saldo final, pero de ANTES de sumarle la cartera. No hace falta (ni corresponde) restarle nada al saldo final: la cartera no se resta, se suma una sola vez, y ese paso ya está hecho.

9. Referencia rápida: función → archivo → qué hace

Función	Qué calcula
_primer_libro_de_caja	El libro más viejo de la caja — el único cuyo saldo_inicial sirve de base
_ultimo_libro_de_caja	El libro vigente — sólo para MOSTRAR en pantalla, ya no se usa para calcular
_movimientos_firmes_de_caja	Todos los movimientos firmes de la caja (cualquier libro, sin diferido pendiente)
_movimientos_firmes_excluyendo_cartera	Igual a la anterior, pero sin contar los cheques en cartera (para no duplicarlos)
_cheques_en_cartera_qs / _cheques_en_cartera	Cheques para entregar a un productor, todavía sin entregar; el segundo ya devuelve el total con el signo a favor nuestro
_movimientos_diferidos_pendientes	Diferidos futuros de la caja, agrupados por fecha, para la proyección
_calcular_estado_caja	Arma el resultado completo de la pantalla "Calcular" genérica
_saldo_base_defecto	Arma los renglones "Movimientos" + "Cheques en cartera" de "por defecto"
_agregar_proyeccion_diferido	Le agrega a "por defecto" los renglones de fechas futuras
_calcular_estado_caja_defecto	Junta Macro + Nación + Global para toda la pantalla "por defecto"

Todo vive en `movimientos_caja/views.py`. El comando de sólo lectura `python manage.py auditar_estado_caja_libro --caja <Macro|Nación>` sirve para ver, línea por línea, qué movimientos concretos está sumando cada parte del cálculo en la base real — útil para verificar cualquier duda futura sin tener que leer código.